

FastMCP Documentation Questions

MCP Assignment

1. FastMCP Transports: stdio, HTTP, and SSE

FastMCP supports three transports: stdio, HTTP, and SSE. Read the deployment docs and explain when you'd pick stdio, HTTP, and what actually changes in your `mcp.run()` call to switch between them.

I'd pick stdio when the server is only for me on my own computer. It's the default transport. The client (the Inspector) starts my server as a subprocess and communicates with it via the process's input and output, so there's no URL, port, or hosting to worry about. That's why I used it for my job finder, since I'm the only user.

I'd pick HTTP (the streamable HTTP transport) when the server needs to be reachable via a URL, for example when it's deployed on a cloud server or shared by several clients at the same time. With stdio, each client gets its own process, but an HTTP server keeps running and serves many clients. If I wanted a public link that other people could connect to, I'd have to host it somewhere and put it behind HTTPS.

SSE is the older web transport. The docs call it legacy and recommend HTTP for all new projects, so I'd only use SSE to support an old client that can't use streamable HTTP yet.

2. Structured Output vs Plain Text

FastMCP automatically turns some tool return values into "structured output" and others into plain text. Read the tools docs and explain the difference between what happens when a tool returns a plain string vs when it returns a dict or a Pydantic model, and why that distinction would matter to a client calling your tool.

When a tool returns a plain string, the client gets it as a normal text content block, just text. That's what my `search_jobs` does, because it returns one big formatted string. That works fine when Claude is the one reading it.

When a tool returns a dict, dataclass or Pydantic model, FastMCP turns it into structured content, which is machine-readable JSON, and the docs say this always happens for object-like results even without a schema. FastMCP can also build an output schema from the function's return type annotation, so the client knows the exact shape of the result and it gets validated. For simple types like `str` or `int`, a generated schema wraps the value under a `result` key so it's still a valid object.

This matters to a client because of how it reads the result. With plain text, a program has to pull the data out of the string itself, like finding the line that starts with "Salary:", and that breaks if I change the wording. With structured output, it can read fields directly, like `job["salary"]` or `job["link"]`, so it can sort jobs by salary, filter them or show them as cards. For a chat assistant, text is enough. For an app that calls my tool in code, structured output is safer.

If I continued this project, I'd change `search_jobs` to return a dict or Pydantic model with a list of jobs (title, company, salary, link) instead of one long string.

MCP Assignment

MCP Assignment

1. Difference Between a Tool, a Resource, and a Prompt in MCP

Explain the difference between a tool, a resource, and a prompt in MCP, with a one-line example of when you'd reach for each.

In MCP, these are the three things a server can offer, and each one has a different job.

A **tool** is an action. It is a function that the AI decides to call, and it actually does something, like fetching data from the internet or changing something. Example: in my project, I use a tool (`search_jobs`) when I want to fetch live job vacancies for a role.

A **resource** is read-only information. It is data sitting at a URI that the client can read, like opening a file. It doesn't do anything and has no side effects. Example: I use a resource (`data://job-sources`) when I want to give the AI background info, like where my job data comes from and what its limits are.

A **prompt** is a ready-made instruction template that the user picks on purpose, and it can take arguments. It doesn't run code or fetch data, it just gives the AI a well-written request. Example: I use a prompt (`job_search_prompt`) when I want a consistent way to ask "find me jobs for this role at my experience level and tell me which to apply to first".

A simple way to remember it: a tool does, a resource informs, and a prompt guides. The model usually decides when to use a tool, the app decides when to read a resource, and the user decides when to use a prompt.

2. Why a Local MCP Server Uses stdio with Claude Desktop

A local MCP server usually talks to Claude Desktop over stdio instead of a normal HTTP API. What does that mean in practice, and why does it make sense for this use case?

With stdio (standard input/output), there is no web address and no network port involved. Claude Desktop starts my server as a program on my own computer, using the command I put in its config file, and then the two talk by passing messages through the program's input and output streams. The client writes a request into the server's input, and the server writes its reply to its output. It's like two programs passing notes directly to each other, not sending letters through the internet.